

# BOOKVAULT

## Marketplace Litteraire Numerique

Cahier des Charges Fonctionnel | Frontend & Backend

|                 |                                                               |
|-----------------|---------------------------------------------------------------|
| Projet          | BookVault - Plateforme E-Book & Livres Physiques              |
| Version         | 1.0.0 - Initial Release                                       |
| Stack Backend   | Spring Boot 3.x   Spring Security   JPA   PostgreSQL   Lombok |
| Stack Frontend  | Angular   TypeScript   RxJS   TailwindCSS                     |
| Base de Donnees | PostgreSQL (Relationnel)   JPA/Hibernate (ORM)                |

## 1. Introduction et Contexte du Projet

BookVault est une marketplace litteraire permettant aux utilisateurs de telecharger, vendre et acheter des livres numeriques (e-books) et physiques. La plateforme vise a connecter auteurs, editeurs et lecteurs dans un ecosysteme fluide, securise et evolutif.

### 1.1 Objectifs Principaux

- Offrir une experience d'achat et de lecture de livres en ligne (numeriques et physiques).
- Permettre aux auteurs et editeurs de publier et vendre leurs oeuvres directement.
- Integrer un systeme de gestion des utilisateurs avec des roles distincts (Lecteur, Auteur, Administrateur).
- Fournir une API REST robuste, documentee et securisee comme colonne vertebrale du systeme.
- Garantir la scalabilite, la maintenabilite et la conformite aux standards OWASP.

### 1.2 Acteurs du Systeme

| Service / Endpoint      | Description & Responsibility                                                                               |
|-------------------------|------------------------------------------------------------------------------------------------------------|
| Visiteur (anonyme)      | Navigation catalogue, recherche livres, consultation fiches produits, inscription.                         |
| Lecteur (USER)          | Achat, telechargement e-books, historique commandes, gestion profil, liste de souhaits, avis.              |
| Auteur/Vendeur (AUTHOR) | Publication livres, gestion catalogue personnel, consultation statistiques ventes, gestion fichiers.       |
| Administrateur (ADMIN)  | Gestion globale utilisateurs, validation contenus, supervision commandes, configuration systeme, rapports. |

## 2. Services Backend - API REST Spring Boot

Le backend est structure en modules fonctionnels independants, chacun exposant des endpoints REST. Tous les services s'appuient sur Spring Web pour le routage, Spring Data JPA pour la persistance, Spring Security pour l'autorisation, et Validation pour la verification des entrees.

### 2.1 Service Authentication & Autorisation

**Module : AuthController | /api/v1/auth/\*\***

Gere l'inscription, la connexion, la deconnexion et la gestion des sessions/tokens JWT. Spring Security fournit les filtres d'authentification et la gestion des roles (RBAC).

| Service / Endpoint         | Description & Responsibility                                                                                                                                 |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| POST /auth/register        | Inscription nouvel utilisateur. Validation des champs (email unique, mot de passe fort). Encodage BCrypt du mot de passe. Attribution role USER par default. |
| POST /auth/login           | Authentification par email/mot de passe. Generation token JWT (access + refresh). Retourne les roles et permissions de l'utilisateur.                        |
| POST /auth/refresh         | Renouvellement du token d'acces a partir du refresh token. Verification validee et non-revocation.                                                           |
| POST /auth/logout          | Invalidation du token actif (blacklist JWT). Nettoyage session cote serveur.                                                                                 |
| POST /auth/forgot-password | Envoi email de reinitialisation mot de passe avec token temporaire (expiration 15 min).                                                                      |
| POST /auth/reset-password  | Reinitialisation effective du mot de passe via token valide. Invalidation de tous les tokens actifs.                                                         |
| GET /auth/me               | Retourne le profil complet de l'utilisateur authentifie (roles, donnees, preferences).                                                                       |

## 2.2 Service Gestion des Utilisateurs

### Module : UserController | /api/v1/users/\*\*

Gestion CRUD des profils utilisateurs avec controle d'accès basé sur les rôles. Seul un ADMIN peut lister tous les utilisateurs. Chaque utilisateur gère son propre profil.

| Service / Endpoint      | Description & Responsibility                                                                       |
|-------------------------|----------------------------------------------------------------------------------------------------|
| GET /users              | Liste paginée de tous les utilisateurs. ADMIN uniquement. Filtres: rôle, statut, date inscription. |
| GET /users/{id}         | Détails d'un utilisateur spécifique. Accès: propriétaire du compte ou ADMIN.                       |
| PUT /users/{id}         | Mise à jour profil (nom, avatar, bio, préférences). Validation des entrées via @Valid.             |
| DELETE /users/{id}      | Désactivation/suppression compte. Anonymisation données (RGPD). ADMIN ou propriétaire.             |
| GET /users/{id}/orders  | Historique commandes de l'utilisateur. Page et triable par date/statut.                            |
| GET /users/{id}/library | Bibliothèque personnelle: livres achetés/téléchargeables. Filtres par format et catégorie.         |
| PUT /users/{id}/role    | Changement de rôle utilisateur (USER -> AUTHOR, ADMIN). ADMIN uniquement.                          |

## 2.3 Service Catalogue Livres

### Module : BookController | /api/v1/books/\*\*

Cœur du système. Gère la création, modification, publication et consultation des livres. Intègre Spring Data JPA pour les requêtes complexes (tri, filtrage, pagination via Pageable).

| Service / Endpoint        | Description & Responsibility                                                                                                                    |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| GET /books                | Catalogue public paginé. Filtres: catégorie, auteur, prix, format, langue, note. Tri: popularité, date, prix. Accessible sans authentification. |
| GET /books/{id}           | Fiche complète d'un livre: métadonnées, auteur, prix, formats disponibles, avis, stock. Incrément compteur vues.                                |
| POST /books               | Création d'un nouveau livre par un AUTHOR ou ADMIN. Validation: titre, description, prix, catégorie obligatoires.                               |
| PUT /books/{id}           | Mise à jour métadonnées livre. Seul le propriétaire (AUTHOR) ou ADMIN peut modifier.                                                            |
| DELETE /books/{id}        | Suppression logique du livre (soft delete). Préservation historique commandes.                                                                  |
| PATCH /books/{id}/publish | Publication/dépublication d'un livre. Changement statut DRAFT -> PUBLISHED. Validation complétude fiche.                                        |

| Service / Endpoint      | Description & Responsibility                                                                     |
|-------------------------|--------------------------------------------------------------------------------------------------|
| GET /books/{id}/preview | Acces aux premieres pages de l'e-book (extrait gratuit). Genere URL signee temporaire.           |
| GET /books/search       | Recherche full-text par titre, auteur, ISBN, mots-cles. Supporte recherche partielle et accents. |
| GET /books/bestsellers  | Top livres les plus vendus sur une periode donnee (7j, 30j, tout). Calcul dynamique.             |
| GET /books/recommended  | Recommandations personnalisees basees sur historique achat utilisateur connecte.                 |

## 2.4 Service Categories & Taxonomie

Module : CategoryController | /api/v1/categories/\*\*

Gestion de la structure hierarchique des categories de livres. Permet une navigation facettee efficace du catalogue.

| Service / Endpoint         | Description & Responsibility                                                         |
|----------------------------|--------------------------------------------------------------------------------------|
| GET /categories            | Arbre complet des categories avec sous-categories et compteur livres associes.       |
| GET /categories/{id}/books | Liste des livres appartenant a une categorie, pages et filtres.                      |
| POST /categories           | Creation d'une nouvelle categorie (ADMIN). Nom, slug, description, parent optionnel. |
| PUT /categories/{id}       | Modification d'une categorie existante: nom, icone, parent, ordre d'affichage.       |
| DELETE /categories/{id}    | Suppression categorie si vide (aucun livre associe). Verification contraintes.       |

## 2.5 Service Gestion des Fichiers & Media

Module : FileController | /api/v1/files/\*\*

Gestion de l'upload, stockage et telechargement securise des fichiers e-book (PDF, EPUB, MOBI) et des images (couvertures, avatars). Integration avec stockage objet (S3/MinIO ou systeme local).

| Service / Endpoint       | Description & Responsibility                                                                                             |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------|
| POST /files/upload/ebook | Upload du fichier e-book (PDF/EPUB/MOBI). Validation type MIME, taille max (50MB). Stockage securise. AUTHOR uniquement. |

| Service / Endpoint                 | Description & Responsibility                                                                                       |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| POST /files/upload/cover           | Upload image de couverture. Redimensionnement automatique (thumbnails multiples). Formats: JPG, PNG, WEBP.         |
| GET /files/ebook/{bookId}/download | Telechargement e-book apres verification achat. Generation URL signee temporaire (30 min). Watermarking optionnel. |
| POST /files/upload/avatar          | Upload avatar utilisateur. Redimensionnement 200x200px. Remplacement ancien avatar.                                |
| DELETE /files/{fileId}             | Suppression fichier du stockage. Verification droits proprietaire ou ADMIN.                                        |

## 2.6 Service Commandes & Paiement

### Module : OrderController | /api/v1/orders/\*\*

Gestion du cycle de vie complet des commandes: creation panier, validation, paiement, fulfillment et suivi. Integration avec gateway de paiement externe (Stripe/PayPal) via webhooks.

| Service / Endpoint       | Description & Responsibility                                                                                        |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|
| POST /orders             | Creation d'une nouvelle commande depuis le panier. Verification disponibilite, calcul total TTC, reservation stock. |
| GET /orders/{id}         | Details complets d'une commande: articles, prix, statut, livraison, facture. Acces proprietaire ou ADMIN.           |
| GET /orders              | Liste toutes les commandes (ADMIN) ou commandes de l'utilisateur connecte. Filtres statut, date, montant.           |
| POST /orders/{id}/pay    | Initiation paiement. Retourne session/URL Stripe ou PayPal. Enregistrement tentative paiement.                      |
| POST /orders/webhook     | Reception webhooks paiement (Stripe/PayPal). Verification signature HMAC. Mise a jour statut commande.              |
| POST /orders/{id}/cancel | Annulation commande (si statut PENDING). Remboursement automatique si paiement effectue.                            |
| GET /orders/{id}/invoice | Generation et telechargement facture PDF de la commande. Numerotation automatique.                                  |
| POST /cart/add           | Ajout article au panier (session ou base de donnees). Verification stock disponible.                                |
| DELETE /cart/{itemId}    | Suppression article du panier. Mise a jour totaux.                                                                  |
| GET /cart                | Consultation panier actuel avec prix mis a jour et disponibilite verifiee.                                          |

## 2.7 Service Avis & Notations

### Module : ReviewController | /api/v1/reviews/\*\*

Gestion des avis et notes laisses par les lecteurs. Seuls les acheteurs verifiés peuvent noter un livre. Systeme de moderation par les administrateurs.

| Service / Endpoint         | Description & Responsibility                                                                                            |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| GET /books/{id}/reviews    | Liste paginee des avis d'un livre. Tri: date, note, utilite. Inclut note moyenne et repartition etoiles.                |
| POST /books/{id}/reviews   | Soumission d'un avis (note 1-5 + commentaire). Verification achat prealable obligatoire. Un avis par livre/utilisateur. |
| PUT /reviews/{id}          | Modification d'un avis existant par son auteur. Historique modification conserve.                                       |
| DELETE /reviews/{id}       | Suppression avis par proprietaire ou ADMIN (moderation).                                                                |
| POST /reviews/{id}/helpful | Marquer un avis comme utile (vote). Un vote par utilisateur par avis. Calcul score utile.                               |
| POST /reviews/{id}/report  | Signalement d'un avis inapproprié pour moderation. Notification ADMIN.                                                  |

## 2.8 Service Auteurs & Profils Publics

Module : AuthorController | /api/v1/authors/\*\*

Gestion des profils auteurs, de leurs catalogues et statistiques de vente. Les auteurs disposent d'un tableau de bord dedie avec analytics.

| Service / Endpoint        | Description & Responsibility                                                             |
|---------------------------|------------------------------------------------------------------------------------------|
| GET /authors              | Liste paginee des auteurs de la plateforme. Filtres: specialite, pays, popularite.       |
| GET /authors/{id}         | Profil public complet d'un auteur: bio, photo, catalogue, statistiques publiques, avis.  |
| GET /authors/{id}/books   | Catalogue complet d'un auteur specifique, pagine et triable.                             |
| GET /authors/me/dashboard | Tableau de bord auteur: ventes totales, revenus, livres populaires, evolution mensuelle. |
| GET /authors/me/stats     | Statistiques detaillees: telechargements, ventes par livre, taux conversion, avis recus. |
| PUT /authors/me/profile   | Mise a jour profil auteur: bio, reseaux sociaux, site web, specialites.                  |

## 2.9 Service Liste de Souhaits

Module : WishlistController | /api/v1/wishlist/\*\*

Permet aux utilisateurs de sauvegarder des livres pour un achat ulterieur. Notifications de baisses de prix et promotions sur les articles en liste de souhaits.

| Service / Endpoint          | Description & Responsibility                                                    |
|-----------------------------|---------------------------------------------------------------------------------|
| GET /wishlist               | Liste de souhaits de l'utilisateur connecte avec prix actuels et disponibilite. |
| POST /wishlist/{bookId}     | Ajout d'un livre a la liste de souhaits. Doubleton ignore silencieusement.      |
| DELETE /wishlist/{bookId}   | Suppression d'un livre de la liste de souhaits.                                 |
| POST /wishlist/move-to-cart | Transfert de tout ou partie de la liste de souhaits vers le panier.             |

## 2.10 Service Notifications

**Module : NotificationController | /api/v1/notifications/\*\***

Système de notifications in-app et email pour informer les utilisateurs des evenements importants (confirmation commande, nouveau livre auteur suivi, promotion, etc.).

| Service / Endpoint             | Description & Responsibility                                                       |
|--------------------------------|------------------------------------------------------------------------------------|
| GET /notifications             | Liste des notifications de l'utilisateur (lues et non lues). Pagine, tri par date. |
| PATCH /notifications/{id}/read | Marquer une notification comme lue. Mise a jour compteur badge.                    |
| PATCH /notifications/read-all  | Marquer toutes les notifications comme lues.                                       |
| DELETE /notifications/{id}     | Suppression d'une notification specifique.                                         |
| PUT /notifications/preferences | Configuration preferences: canaux (email, in-app), types d'evenements a recevoir.  |

## 2.11 Service Administration

**Module : AdminController | /api/v1/admin/\*\***

Ensemble des endpoints reserves aux administrateurs pour la supervision globale de la plateforme, la moderation des contenus et la configuration systeme.

| Service / Endpoint   | Description & Responsibility                                                                |
|----------------------|---------------------------------------------------------------------------------------------|
| GET /admin/dashboard | Vue globale: nombre utilisateurs, commandes du jour, revenus, livres en attente validation. |

| Service / Endpoint                               | Description & Responsibility                                                                          |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>GET</b><br><b>/admin/books/pending</b>        | Livres en attente de validation avant publication. File de moderation.                                |
| <b>PATCH</b><br><b>/admin/books/{id}/approve</b> | Validation/Rejection d'un livre soumis par un auteur. Notification auteur par email.                  |
| <b>GET /admin/users</b>                          | Gestion complete utilisateurs: recherche, filtres, modification roles, suspension/activation comptes. |
| <b>GET /admin/orders</b>                         | Supervision toutes les commandes. Filtres avances, export CSV, gestion litiges.                       |
| <b>GET /admin/reports/sales</b>                  | Rapports de vente: revenus par periode, par categorie, par auteur. Export Excel/CSV.                  |
| <b>GET /admin/reports/users</b>                  | Rapports utilisateurs: inscriptions, retention, activite. Metriques cles.                             |
| <b>POST /admin/promotions</b>                    | Creation codes promo et promotions: pourcentage/montant fixe, duree, conditions.                      |
| <b>GET /admin/logs</b>                           | Consultation logs applicatifs et journal d'audit des actions sensibles.                               |

### 3. Services Frontend - Angular SPA

Le frontend est une Single Page Application Angular structuree en modules fonctionnels. Chaque module encapsule ses composants, services, guards et modeles de donnees. La communication avec le backend s'effectue exclusivement via des services Angular injectables consommant l'API REST.

#### 3.1 Module Authentification (AuthModule)

Gestion complete du cycle d'authentification utilisateur cote client. Integration avec les tokens JWT, intercepteurs HTTP et protection des routes.

| Service / Endpoint           | Description & Responsibility                                                                                                                     |
|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AuthService</b>           | Service central: login(), register(), logout(), refreshToken(). Stockage securise JWT (HttpOnly cookie prefere). Gestion expiration automatique. |
| <b>AuthGuard / RoleGuard</b> | Guards Angular: protection routes par authentification et role (canActivate). Redirection vers /login si non autorise.                           |
| <b>LoginComponent</b>        | Formulaire connexion avec validation reactive. Gestion erreurs API. Remember me. Lien mot de passe oublie.                                       |
| <b>RegisterComponent</b>     | Formulaire inscription multi-etapes. Validation synchrone/asynchrone (email unique). Confirmation email.                                         |



| Service / Endpoint             | Description & Responsibility                                                                             |
|--------------------------------|----------------------------------------------------------------------------------------------------------|
| <b>ForgotPasswordComponent</b> | Formulaire demande reinitialisation. Confirmation envoi email. Timer re-envoi.                           |
| <b>JwtInterceptor</b>          | Intercepteur HTTP: ajout automatique header Authorization a chaque requete. Gestion 401 -> refresh auto. |
| <b>AuthStateService</b>        | Gestion etat utilisateur via BehaviorSubject. currentUser\$, isLoggedIn\$, userRoles\$ observables.      |

### 3.2 Module Catalogue & Navigation (CatalogModule)

Experience de navigation et decouverte du catalogue livres. Optimise pour la performance avec pagination virtuelle et lazy loading des images.

| Service / Endpoint              | Description & Responsibility                                                                                  |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>BookListComponent</b>        | Grille/liste livres paginee. Toggle vue grille/liste. Tri et filtres persistants en URL queryParams.          |
| <b>BookCardComponent</b>        | Carte livre reutilisable: couverture, titre, auteur, prix, note, badge (Nouveau/Bestseller). Actions rapides. |
| <b>BookDetailComponent</b>      | Page fiche complete: galerie images, description, formats disponibles, avis, auteur, livres similaires.       |
| <b>SearchComponent</b>          | Barre recherche avec autocompletion (debounce 300ms). Historique recherches recentes. Suggestions.            |
| <b>FilterSidebarComponent</b>   | Panneau filtres: categories arborescentes, fourchette prix (slider), format, langue, note minimale.           |
| <b>CategoryBrowserComponent</b> | Navigation par categories avec breadcrumb. Affichage hierarchique et compteurs.                               |
| <b>BestsellersComponent</b>     | Section livres populaires avec onglets periode (Semaine/Mois/Tout temps).                                     |
| <b>BookService</b>              | Service Angular: getAllBooks(), getBook(id), searchBooks(), getRecommended(). Gestion cache RxJS.             |

### 3.3 Module Panier & Commande (ShopModule)

Gestion du tunnel d'achat de bout en bout. Synchronisation panier local/serveur. Integration gateway paiement via redirection securisee.

| Service / Endpoint   | Description & Responsibility                                                                            |
|----------------------|---------------------------------------------------------------------------------------------------------|
| <b>CartComponent</b> | Page panier: liste articles, quantites, sous-total, application code promo, total TTC. Bouton commande. |

| Service / Endpoint                | Description & Responsibility                                                                                                              |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| <b>CartService</b>                | Gestion etat panier via BehaviorSubject. addItem(), removeItem(), clearCart(), syncWithServer(). Persistance localStorage pour visiteurs. |
| <b>CartIconComponent</b>          | Icône panier header avec badge compteur. Mise a jour temps reel via observable.                                                           |
| <b>CheckoutComponent</b>          | Page commande: recapitulatif, choix livraison (physique), saisie adresse, selection paiement.                                             |
| <b>PaymentComponent</b>           | Integration Stripe Elements ou PayPal SDK. Validation PCI-DSS. Gestion 3DS.                                                               |
| <b>OrderConfirmationComponent</b> | Page confirmation apres paiement reussi. Details commande, liens telechargement e-books, email confirme.                                  |
| <b>OrderService</b>               | Service commandes: createOrder(), getOrders(), getOrder(id), cancelOrder(). Polling statut paiement.                                      |

### 3.4 Module Espace Utilisateur (UserModule)

Dashboard personnel du lecteur: gestion profil, bibliotheque, historique et preferences. Routes proteges par AuthGuard.

| Service / Endpoint            | Description & Responsibility                                                                                |
|-------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>ProfileComponent</b>       | Formulaire edition profil: photo, nom, email, mot de passe, preferences langue. Upload avatar avec preview. |
| <b>LibraryComponent</b>       | Bibliotheque e-books achetes. Grille avec liens telechargement directs et historique acces.                 |
| <b>OrderHistoryComponent</b>  | Historique commandes pagine. Statuts colores. Liens factures PDF. Details expandables.                      |
| <b>WishlistComponent</b>      | Liste de souhaits: gestion ajout/suppression, transfer vers panier, alertes prix.                           |
| <b>NotificationsComponent</b> | Centre notifications: liste temps reel via SSE ou polling, marquage lecture, preferences.                   |
| <b>UserService</b>            | Service donnees utilisateur: updateProfile(), getLibrary(), getOrders(), getWishlist().                     |

### 3.5 Module Auteur (AuthorModule)

Espace dedie aux auteurs/vendeurs. Acces conditionne au role AUTHOR. Dashboard analytics et outils de publication.

| Service / Endpoint              | Description & Responsibility                                                                           |
|---------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>AuthorDashboardComponent</b> | Vue d'ensemble: KPIs ventes, revenus du mois, livres en ligne, avis recents. Graphiques Chart.js.      |
| <b>BookManagementComponent</b>  | Tableau gestion livres: liste, statuts (Brouillon/Publie/En attente), actions rapides.                 |
| <b>BookFormComponent</b>        | Formulaire creation/edition livre multi-etapes: informations, upload fichiers, prix, couverture, tags. |
| <b>FileUploadComponent</b>      | Composant upload drag-and-drop: barre progression, validation type/taille, preview couverture.         |
| <b>SalesStatsComponent</b>      | Statistiques ventes: graphiques revenus temporels, classement livres, taux de conversion.              |
| <b>AuthorService</b>            | Service auteur: createBook(), updateBook(), uploadFile(), getStats(), getDashboard().                  |

### 3.6 Module Administration (AdminModule)

Interface d'administration complete. Acces exclusif role ADMIN. Lazy-loaded pour performance.

| Service / Endpoint                  | Description & Responsibility                                                                  |
|-------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>AdminDashboardComponent</b>      | Vue superviseur: metriques globales temps reel, alertes, livres en attente, activite recente. |
| <b>UserManagementComponent</b>      | Tableau utilisateurs: recherche, filtres, modification roles inline, suspension, export CSV.  |
| <b>BookModerationComponent</b>      | File d'attente moderation: apercu livre, approbation/rejet avec motif, notification auteur.   |
| <b>OrderManagementComponent</b>     | Gestion commandes globale: filtres avances, mise a jour statuts, gestion remboursements.      |
| <b>PromotionManagementComponent</b> | CRUD codes promo: type, montant, valideite, usage limit, conditions d'eligibilite.            |
| <b>AdminService</b>                 | Service admin: getStats(), moderateBook(), manageUsers(), getReports(), createPromotion().    |

### 3.7 Services Transversaux Frontend

Services partages entre tous les modules, injectes au niveau root pour un acces global.

| Service / Endpoint | Description & Responsibility                                                                             |
|--------------------|----------------------------------------------------------------------------------------------------------|
| <b>ApiService</b>  | Service HTTP base: gestion URL, headers, retry automatique, serialisation/deserialisation JSON uniforme. |

| Service / Endpoint         | Description & Responsibility                                                                             |
|----------------------------|----------------------------------------------------------------------------------------------------------|
| <b>ErrorHandlerService</b> | Intercepteur erreurs global: traduction codes HTTP en messages utilisateur, logging, notification toast. |
| <b>ToastService</b>        | Notifications toast non-bloquantes: succes, erreur, info, warning. File d'attente et auto-dismiss.       |
| <b>LoadingService</b>      | Etat chargement global via BehaviorSubject. Spinner overlay conditionnel. Blocage navigation.            |
| <b>ThemeService</b>        | Gestion theme clair/sombre. Persistance preference. Detection systeme auto.                              |
| <b>LocalStorageService</b> | Abstraction stockage local typee. Serialisation securisee. Gestion expiration.                           |
| <b>SeoService</b>          | Mise a jour dynamique meta tags (Title, Description, OpenGraph) pour chaque route.                       |

## 4. Dependances Backend & Leur Role

Chaque dependance Spring Boot repond a un besoin fonctionnel precis au sein de la plateforme BookVault. Le tableau ci-dessous detaille la responsabilite de chacune.

| Dependency               | Category                     | Role in Platform                                                                                                                                                        |
|--------------------------|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Spring Web</b>        | <a href="#">API Layer</a>    | Expose tous les endpoints REST (BookController, AuthController, etc.). Gestion routage, serialisation JSON (Jackson), gestion requetes HTTP multipart pour les uploads. |
| <b>Spring Data JPA</b>   | <a href="#">Persistence</a>  | ORM avec Hibernate. Repositories pour Book, User, Order, Review. Requetes JPQL, pagination native via Pageable, relations @OneToMany, @ManyToMany entre entites.        |
| <b>PostgreSQL Driver</b> | <a href="#">Database</a>     | Connecteur JDBC PostgreSQL. Gestion transactions ACID. Support types natifs PG (UUID, JSONB). Pool de connexions via HikariCP inclus.                                   |
| <b>Lombok</b>            | <a href="#">Code Quality</a> | Reduction boilerplate: @Data, @Builder sur les entites JPA et DTOs. @RequiredArgsConstructor pour injection dependances. @Slf4j pour logging uniforme.                  |
| <b>Spring Security</b>   | <a href="#">Security</a>     | Filtre JWT stateless, RBAC (USER/AUTHOR/ADMIN), encodage BCrypt mots de passe, CORS configuration,                                                                      |

| Dependency           | Category       | Role in Platform                                                                                                                                                          |
|----------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      |                | protection CSRF, securisation endpoints par role avec @PreAuthorize.                                                                                                      |
| Spring Boot DevTools | Developer XP   | Hot reload automatique en developpement. Rechargement contexte Spring sans redemarrage. Configurations specifiques profil dev (H2 optionnel, logs verbose).               |
| Validation (Jakarta) | Data Integrity | @Valid, @NotBlank, @Email, @Size, @Min/@Max, @Pattern sur les DTOs de requete. BindingResult et @ControllerAdvice pour reponses d'erreur structurees 400.                 |
| Spring HATEOAS       | API Design     | Enrichissement reponses REST avec liens hypermedia (self, next, prev). EntityModel, CollectionModel, PagedModel pour navigation API plus riche et auto-documentee.        |
| OpenAPI / Swagger    | Documentation  | Documentation API auto-generee via annotations @Operation, @ApiResponse, @Parameter. Interface Swagger UI interactive sur /swagger-ui.html. Export spec OpenAPI 3.0 JSON. |

## 5. Architecture Securite

### 5.1 Modele de Securite JWT

- Access Token: JWT signe (HS256/RS256), expiration courte (15-60 min). Contient userId, email, roles[].
- Refresh Token: Token opaque stocke en base de donnees, longue duree (7-30 jours). Rotation a chaque usage.
- Blacklisting: Table token\_blacklist pour invalidation immediate (logout, changement mdp, suspension).
- JwtAuthenticationFilter: Filtre Spring Security extrayant et validant le JWT a chaque requete protegee.

### 5.2 Controle d'Acces par Role (RBAC)

| Service / Endpoint | Description & Responsibility                                                                                    |
|--------------------|-----------------------------------------------------------------------------------------------------------------|
| ROLE_USER          | Achat livres, lecture bibliotheque, depot avis (acheteur verifie), gestion profil personnel, liste souhaits.    |
| ROLE_AUTHOR        | Toutes permissions USER + publication livres, gestion catalogue personnel, acces dashboard statistiques ventes. |

| Service / Endpoint | Description & Responsibility                                                                                           |
|--------------------|------------------------------------------------------------------------------------------------------------------------|
| ROLE_ADMIN         | Toutes permissions AUTHOR + administration globale utilisateurs, moderation contenus, rapports, configuration systeme. |

### 5.3 Bonnes Pratiques OWASP

- Validation stricte toutes entrees (@Valid + sanitisation XSS).
- Requetes parametrees via JPA (prevention SQL Injection).
- Rate limiting sur endpoints sensibles (auth, upload).
- CORS configuration restrictive (origines autorisees explicites).
- Headers securite: HSTS, X-Frame-Options, Content-Security-Policy.
- Audit log toutes actions sensibles (connexions, modifications roles, suppressions).

## 6. Entites Principales (Modele de Donnees)

Le schema relationnel PostgreSQL est genere par Hibernate via les annotations JPA. Les entites suivantes constituent le coeur du domaine metier BookVault.

| Service / Endpoint | Description & Responsibility                                                                                                                                                                    |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| User               | id (UUID), email (unique), passwordHash, firstName, lastName, role (ENUM), avatarUrl, isActive, createdAt, updatedAt.                                                                           |
| Book               | id (UUID), title, description, isbn, price, format (EBOOK/PHYSICAL/BOTH), coverUrl, fileUrl, status (DRAFT/PUBLISHED/REJECTED), viewCount, author (FK User), category (FK Category), createdAt. |
| Category           | id, name, slug (unique), description, iconUrl, parent (FK Category - auto-reference), displayOrder.                                                                                             |
| Order              | id (UUID), user (FK), status (PENDING/PAID/SHIPPED/DELIVERED/CANCELLED), totalAmount, currency, paymentMethod, paymentReference, createdAt.                                                     |
| OrderItem          | id, order (FK), book (FK), quantity, unitPrice, format. Preservation prix historique.                                                                                                           |
| Review             | id, book (FK), user (FK), rating (1-5), comment, isVerifiedPurchase, helpfulVotes, createdAt. Contrainte unique (book, user).                                                                   |
| Wishlist           | id, user (FK), book (FK), addedAt. Contrainte unique (user, book).                                                                                                                              |
| Notification       | id, user (FK), type (ENUM), title, message, isRead, readAt, relatedEntityId, createdAt.                                                                                                         |
| Promotion          | id, code (unique), type (PERCENTAGE/FIXED), value, minOrderAmount, usageLimit, usageCount, startDate, endDate, isActive.                                                                        |

## 7. Exigences Non-Fonctionnelles

### 7.1 Performance

- Reponse API < 200ms pour 95% des requetes de lecture en charge normale.
- Pagination obligatoire sur tous les endpoints de liste (default: 20 items, max: 100).
- Mise en cache Redis pour catalogue (TTL 5 min) et sessions.
- Lazy loading images et code splitting Angular pour LCP < 2.5s.

### 7.2 Disponibilite & Fiabilite

- SLA cible: 99.9% disponibilite (hors maintenance planifiee).
- Retry automatique transactions paiement avec idempotency keys.
- Gestion degradee: catalogue consultable si service paiement indisponible.

### 7.3 Maintainabilite

- Couverture tests unitaires minimum 80% (JUnit 5 + Mockito backend, Jest + Jasmine frontend).
- Documentation API Swagger complete et a jour a chaque release.
- Logs structures JSON (Logback) avec correlation ID par requete.
- Versioning API: /api/v1/ - Compatibilite retrograde garantie sur version majeure.

## 8. Recapitulatif des Services

| Service / Endpoint         | Description & Responsibility                                                                                                          |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Backend - Modules REST     | 11 controllers (Auth, Users, Books, Categories, Files, Orders, Reviews, Authors, Wishlist, Notifications, Admin)   ~65 endpoints REST |
| Frontend - Modules Angular | 6 modules fonctionnels (Auth, Catalog, Shop, User, Author, Admin) + Services transversaux                                             |
| Services Angular           | 20+ services injectables (ApiService, AuthService, BookService, CartService, OrderService, etc.)                                      |
| Entites JPA/PostgreSQL     | 9 entites principales avec relations et contraintes d'integrite                                                                       |
| Dependances Spring Boot    | 9 starters (Web, JPA, Security, Validation, Lombok, DevTools, PostgreSQL, HATEOAS, OpenAPI)                                           |
| Securite                   | JWT stateless + RBAC 3 roles + OWASP best practices + Audit logging                                                                   |

---

*BookVault | Document genere automatiquement | v1.0.0*